In [1]:

```
# list comprehension
# concise way to create lists
# creates new lists where each element is the result of some operations
# applied to each member of another sequence or iterable, or
# create a subsequence of those elements that satisfy a certain condition.

# newlist = [expression for item in iterable (if condition)]

my_list = ['42', '65', '12']

my_list_int = [int(x) for x in my_list]

print(f"my_list: {my_list}, my_list_int: {my_list_int}")
```

```
my_list: ['42', '65', '12'], my_list_int: [42, 65, 12]
```

In [2]:

```
new_list = [x for x in range(10)]
print(new_list)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [3]:

```
# create a list whose elements are the squares of the numbers from 1 to 10
#21
new_list = [x**2 for x in range(1,11)]
print(new_list)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

In [4]:

```
# create a new list whose values are double the elements of the given list
#43
vec = [-4, -2, 0, 2, 4]
new_list = [2*x for x in vec]
print(new_list)
```

```
[-8, -4, 0, 4, 8]
```

In [5]:

```
# create a new list whose values are the absolute values of the given list
#46
vec = [-4, -2, 0, 2, 4]
new_list = [abs(x) for x in vec]
print(new_list)
```

```
[4, 2, 0, 2, 4]
```

In [6]:

```
# create a list of 2-tuples like (number, square)
# for numbers ranging from 1 to 10
# 33
new_list = [(x,x*x) for x in range(1,11)]
print(new_list)
```

```
[(1, 1), (2, 4), (3, 9), (4, 16), (5, 25), (6, 36), (7, 49), (8, 64), (9, 81), (10, 100)]
```

In [7]:

```
# create a list by filtering out the even numbers
# from the range 0 to 9
```

```
new_list = [x for x in range(10) if x % 2]
print(new_list)
```

```
[1, 3, 5, 7, 9]
```

In [8]:

```
# create a list by filtering out the negative numbers
# from the given list
# 57
vec = [-4, -2, 0, 2, 4]
new_list = [x for x in vec if not x < 0]
print(new_list)
```

```
[0, 2, 4]
```

In [9]:

```
# create a list by squaring the positive numbers
# from the given list

vec = [-4, -2, 0, 2, 4, 5]
new_list = [x*x for x in vec if x > 0]
print(new_list)
```

```
[4, 16, 25]
```

In [10]:

```
# create a list for numbers ranging from 1 to 10,
# which contains square of the number if the number is odd
# and 0 otherwise

new_list = [i*i if i % 2 else 0 for i in range(10)]
print(new_list)
```

```
[0, 1, 0, 9, 0, 25, 0, 49, 0, 81]
```

In [11]:

```
#Creating a matrix Without List Comprehension

# matrix = []
# for i in range(5):
#     # Append an empty sublist inside the list
#     matrix.append([])
#     for j in range(5):
#         matrix[i].append(j)
# print(matrix)


# Nested list comprehension
matrix = [[j for j in range(5)] for i in range(5)]

print(matrix)
```

```
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]
```

In [12]:

```
#Filtering a Nested List Using List Comprehension

#Without List Comprehension
# matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# odd_numbers = []
# for row in matrix:
#     for element in row:
#         if element % 2 != 0:
#             odd_numbers.append(element)

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
odd_numbers = [
    element for row in matrix for element in row if element % 2 != 0]

print(odd_numbers)

print(odd_numbers)
```

```
[1, 3, 5, 7, 9]
[1, 3, 5, 7, 9]
```

In [13]:

```
#Flattening Nested Sub-Lists
# In this example, a 2D list named matrix with varying sublist lengths is flattened using
nested loops.
# The elements from each sublist are sequentially appended to the list flatten_matrix, re
sulting in a flattened representation of the original matrix.

# 2-D List
# matrix = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]

# flatten_matrix = []

# for sublist in matrix:
#  for val in sublist:
#   flatten_matrix.append(val)

# print(flatten_matrix)

# 2-D List
matrix = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]

# Nested List Comprehension to flatten a given 2-D matrix
flatten_matrix = [val for sublist in matrix for val in sublist]

print(flatten_matrix)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [14]:

```
#Manipulate String Using List Comprehension
# Without List Comprehension

# In this example, a 2D list named matrix containing strings is modified using nested loo
ps.
# The inner loop capitalizes the first letter of each fruit, and the outer loop construct
s a new 2D list, modified_matrix,
# with the capitalized fruits, resulting in a matrix of strings with initial capital lett
ers.


matrix = [["apple", "banana", "cherry"],
  ["date", "fig", "grape"],
  ["kiwi", "lemon", "mango"]]

modified_matrix = []
for row in matrix:
 modified_row = []
 for fruit in row:
  modified_row.append(fruit.capitalize())
 modified_matrix.append(modified_row)

print(modified_matrix)


matrix = [["apple", "banana", "cherry"],
  ["date", "fig", "grape"],
  ["kiwi", "lemon", "mango"]]

modified_matrix = [[fruit.capitalize() for fruit in row] for row in matrix]
```

```
print(modified_matrix)
```

```
[['Apple', 'Banana', 'Cherry'], ['Date', 'Fig', 'Grape'], ['Kiwi', 'Lemon', 'Mango']]
[['Apple', 'Banana', 'Cherry'], ['Date', 'Fig', 'Grape'], ['Kiwi', 'Lemon', 'Mango']]
```

In [15]:

```
#Checking Even number Without using set comprehension
# input_list = [1, 2, 3, 4, 4, 5, 6, 6, 6, 7, 7]

# output_set = set()

# for var in input_list:
#  if var % 2 == 0:
#   output_set.add(var)

# print("Output Set using for loop:", output_set)
# Checking Even number using set comprehension
input_list = [1, 2, 3, 4, 4, 5, 6, 6, 6, 7, 7]

set_using_comp = {var for var in input_list if var % 2 == 0}

print("Output Set using set comprehensions:",
      set_using_comp)
```

```
Output Set using set comprehensions: {2, 4, 6}
```

In [16]:

```
# square_dict = dict()
# for num in range(1, 11):
#     square_dict[num] = num*num
# print(square_dict)
#let's create the dictionary in the above program using dictionary comprehension.

#The minimal syntax for dictionary comprehension is:

#dictionary = {key: value   for vars in iterable}
#               num: num*num for num  in range(1, 11)

# dictionary comprehension example
square_dict = {num: num*num for num in range(1, 11)}
print(square_dict)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
```

In [17]:

```
#item price in dollars
old_price = {'milk': 1.02, 'coffee': 2.5, 'bread': 2.5}

dollar_to_pound = 0.76
new_price = {item: value*dollar_to_pound for (item, value) in old_price.items()}
print(new_price)
```

```
{'milk': 0.7752, 'coffee': 1.9, 'bread': 1.9}
```

In [18]:

```
#Example 3: If Conditional Dictionary Comprehension
original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}

even_dict = {k: v for (k, v) in original_dict.items() if v % 2 == 0}
print(even_dict)
```

```
{'jack': 38, 'michael': 48}
```

In [19]:

```
#Example 4: Multiple if Conditional Dictionary Comprehension
original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}
```

```
new_dict = {k: v for (k, v) in original_dict.items() if v % 2 != 0 if v < 40}
print(new_dict)
```

```
{'john': 33}
```

In [20]:

```
#Example 5: if-else Conditional Dictionary Comprehension

original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}

new_dict_1 = {k: ('old' if v > 40 else 'young')
    for (k, v) in original_dict.items()}
print(new_dict_1)
```

```
{'jack': 'young', 'michael': 'old', 'guido': 'old', 'john': 'young'}
```

In [21]:

```
#Example 6: Nested Dictionary with Two Dictionary Comprehensions

dictionary = {
    k1: {k2: k1 * k2 for k2 in range(1, 6)} for k1 in range(2, 5)
}
print(dictionary)

#As you can see, we have constructed a multiplication table in a nested dictionary, for n
umbers from 2 to 4.

#Whenever nested dictionary comprehension is used, Python first starts from the outer loo
p and then goes to the inner one.
```

```
{2: {1: 2, 2: 4, 3: 6, 4: 8, 5: 10}, 3: {1: 3, 2: 6, 3: 9, 4: 12, 5: 15}, 4: {1: 4, 2: 8,
3: 12, 4: 16, 5: 20}}
```

In [22]:

```
#So, the above code would be equivalent to:
dictionary = dict()
for k1 in range(11, 16):
    dictionary[k1] = {k2: k1*k2 for k2 in range(1, 6)}
print(dictionary)

# Unfolded
# dictionary = dict()
# for k1 in range(11, 16):
#     dictionary[k1] = dict()
#     for k2 in range(1, 6):
#         dictionary[k1][k2] = k1*k2
# print(dictionary)
```

```
{11: {1: 11, 2: 22, 3: 33, 4: 44, 5: 55}, 12: {1: 12, 2: 24, 3: 36, 4: 48, 5: 60}, 13: {1:
13, 2: 26, 3: 39, 4: 52, 5: 65}, 14: {1: 14, 2: 28, 3: 42, 4: 56, 5: 70}, 15: {1: 15, 2: 3
0, 3: 45, 4: 60, 5: 75}}
```

In [23]:

```
# In Python, there is no direct tuple comprehension, Python supports list comprehension,
set comprehension, and dictionary comprehension.
# The absence of tuple comprehension is rooted in Python's design philosophy and the char
acteristics of tuples themselves.

# The list comprehensions allow for creating lists dynamically, dictionary comprehensions
for dictionaries, and set comprehensions for sets.
# But we can't have a tuple comprehension because a tuple is an immutable data structure.
The idea of comprehension lies in modifying and filtering data. However,
# we can use list, set, and dictionary comprehension and typecast them into tuples.
```